

Voila! There it is, stating that if a file of type `httpd_exec_t` is executed under a parent process of type `initrc_t`, the resulting process created will have a security context type of `httpd_t`.

As an exercise, let us ensure that the `httpd` process, if executed directly under the shell, should also be created with a security context type of `httpd_t`. To do this, we will need to create a new module and insert it into our policy.

We will need to create a new type enforcement file as discussed in earlier articles. An example of the file is given below:

```
[vbg@vbg-work test-selinux]$ cat http.te
policy_module(http,1.1.0)
#####
#
# Declarations
#
require {
    type httpd_t;
    type unconfined_t;
    type httpd_exec_t;
    role unconfined_r;
    class process transition;
}
#####
#
# http local policy
#
type_transition unconfined_t httpd_exec_t : process httpd_t;
role unconfined_r types httpd_t;
allow unconfined_t httpd_t:process transition;
[vbg@vbg-work test-selinux]$
```

Compile and load this module:

```
[vbg@vbg-work test-selinux]$ make http.pp
[vbg@vbg-work test-selinux]$ su -
[root@vbg-work ~]# semodule -i /home/vbg/test-selinux/http.pp
```

Now execute the command `/usr/sbin/httpd` and note the security context type of the `httpd` process:

```
[root@vbg-work ~]# /usr/sbin/httpd
[root@vbg-work ~]#
[root@vbg-work ~]# ps axZ | grep http
unconfined_u:unconfined_r:httpd_t:s0-c0:c1023 4343 ? Ss 0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:httpd_t:s0-c0:c1023 4345 ? S 0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:httpd_t:s0-c0:c1023 4346 ? S 0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:httpd_t:s0-c0:c1023 4347 ? S 0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:httpd_t:s0-c0:c1023 4350 ? S 0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:httpd_t:s0-c0:c1023 4351 ? S 0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:httpd_t:s0-c0:c1023 4352 ? S 0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:httpd_t:s0-c0:c1023 4354 pts/0 S+ 0:00 grep http
[root@vbg-work ~]#
```

But how is the type of the first process (`init`) set? Does it inherit its type from the kernel? What exactly happens?

The following lines are taken verbatim from Dan Walsh's blog (<http://danwalsh.livejournal.com>): "When the SELinux kernel boots up, it is hard coded to run as `kernel_t`. Since at this point there is no policy running, this is the only context. So since this is the only context, all applications that are run will stay in `kernel_t`. When the kernel executes `/sbin/init` it originally is running as `kernel_t`, then it reads in the policy file and loads it into the kernel. At this point `init` re-execs itself. When `/sbin/init` was installed it is labelled `init_exec_t` and now there is a rule in the kernel that says when `kernel_t` execs an application labelled `init_exec_t`, it should transition to `init_t`. So now the `init` process is running as `init_t`."

Let us check if such a rule really exists in the policy, which applies when `init` re-executes itself:

- The parent process is the kernel.
- The parent process type is `kernel_t`.
- The file being executed is `/sbin/init`.
- The security context type of the file being executed is `init_exec_t`.

```
[root@vbg-work ~]# ls -lZ /sbin/init
-rwxr-xr-x. root root system_u:object_r:init_exec_t:s0 /sbin/init
```

Search the SELinux policy for a type transition rule:

```
root@vbg-work ~]# search -r -c process | grep init_exec_t | grep kernel_t
type_transition kernel_t init_exec_t : process init_t;
```

There we are! When `/sbin/init` is executed under `kernel_t`, the `init` process being created has a type of `init_t`. This can be checked by the following command:

```
[root@vbg-work ~]# pstree -p2 | head -n 1
init(1,'system_u:system_r:init_t:s0')
[root@vbg-work ~]#
```

Now we can understand how subjects or processes created in memory get their security contexts. Applying SELinux policy rules under type enforcement is simply allowing access only to desired objects by the designated subjects, and once we know how to set the security context types of objects and subjects, and how to create allow rules, we can easily achieve the best out of SELinux.

In the next article in this series, we will explore macros and dive deep into them. Do let me know if you have any queries. 

By: Varad Gupta

Varad is an open source enthusiast who strongly believes in the open source collaborative model not only for technology but also for business. India's first RHCSS (Red Hat Certified Security Specialist), he has been involved in spreading open source through Keen & Able Computers Pvt Ltd, an open source systems integration company, and FOSTERING Linux, a FOSS training, education and research training centre. The author can be contacted at varad.gupta@fosteringlinux.com